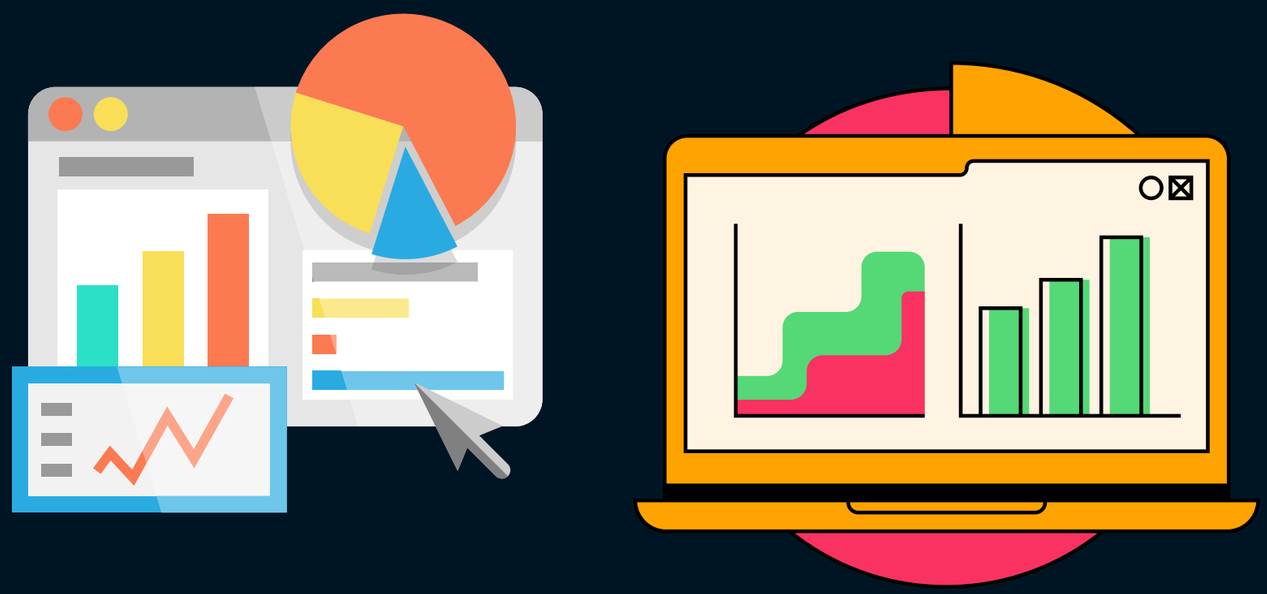
#Matplotlib Comprehensive Guide

AI Basics

```
import matplotlib.pyplot as plt
```



#Python

/sumitkhanna

Python

Follow me on  Sumit Khanna for more updates

Matplotlib Comprehensive Guide

This notebook provides a comprehensive guide to using Matplotlib functions.

Table of Contents

1. Introduction
2. Basic Plotting
    - Line Plot
    - Scatter Plot
    - Bar Plot
    - Histogram
3. Advanced Plotting
    - Pie Chart
    - Box Plot
    - Heatmap
4. Pairwise Charts
5. Statistical Distributions
6. Gridded Charts
7. Irregularly Gridded Charts
8. Styling and Legends
9. Multiple Subplots
10. Multiple Charts in Grid
11. Animated Charts

Introduction

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. This guide will walk you through various Matplotlib functions with practical examples using realistic datasets from banking, fraud detection, and school data.

In [ ]:
```
# Import necessary libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

Basic Plotting

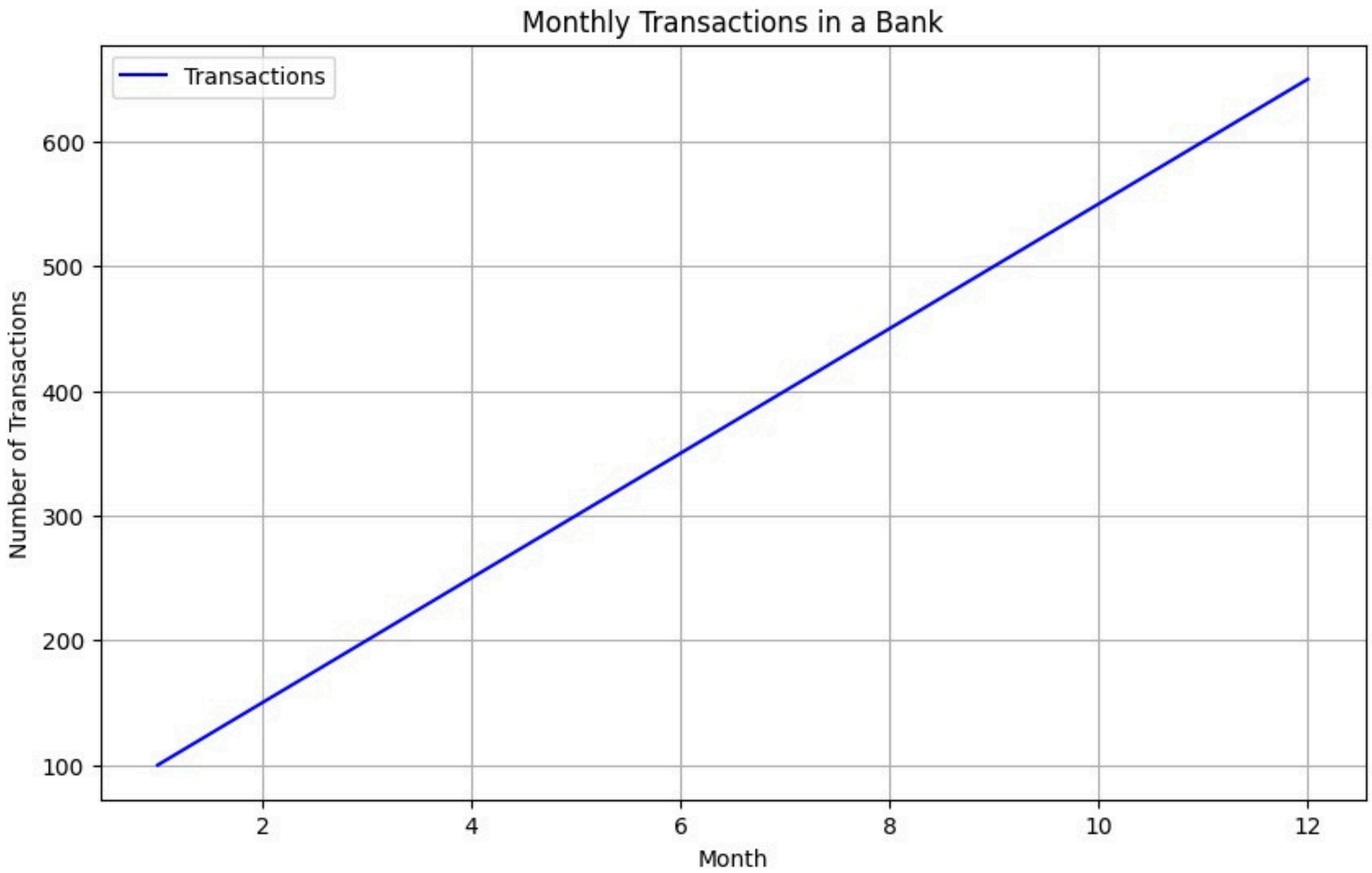
Line Plot

A line plot is one of the most basic and common types of plots. It is used to visualize data points connected by straight lines.

In [ ]:
```
# Line Plot
# Parameters: x (x-axis values), y (y-axis values), label (label for the line), color (color of the line)
# Returns: A line plot

# Example: Monthly transactions in a bank
months = np.arange(1, 13)
transactions = [100, 150, 200, 250, 300, 350, 400, 450, 500, 550, 600, 650]

plt.figure(figsize=(10, 6))
plt.plot(months, transactions, label='Transactions', color='blue')
plt.xlabel('Month')
plt.ylabel('Number of Transactions')
plt.title('Monthly Transactions in a Bank')
plt.legend()
plt.grid(True)
plt.show()
```



Scatter Plot

A scatter plot is used to display values for two variables for a set of data. The data is displayed as a collection of points, each having the value of one variable on the horizontal axis and the value of the other variable on the vertical axis.

In [ ]:
```
# Scatter Plot
# Parameters: x (x-axis values), y (y-axis values), label (label for the points), color (color of the points), marker (style of the points)
# Returns: A scatter plot

# Example: Age vs Salary in a company
age = np.random.randint(20, 60, 50)
salary = age * 1000 + np.random.randint(5000, 20000, 50)
```

```
plt.figure(figsize=(10, 6))
plt.scatter(age, salary, label='Employees', color='red', marker='o')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.title('Age vs Salary in a Company')
plt.legend()
plt.grid(True)
plt.show()
```



Bar Plot

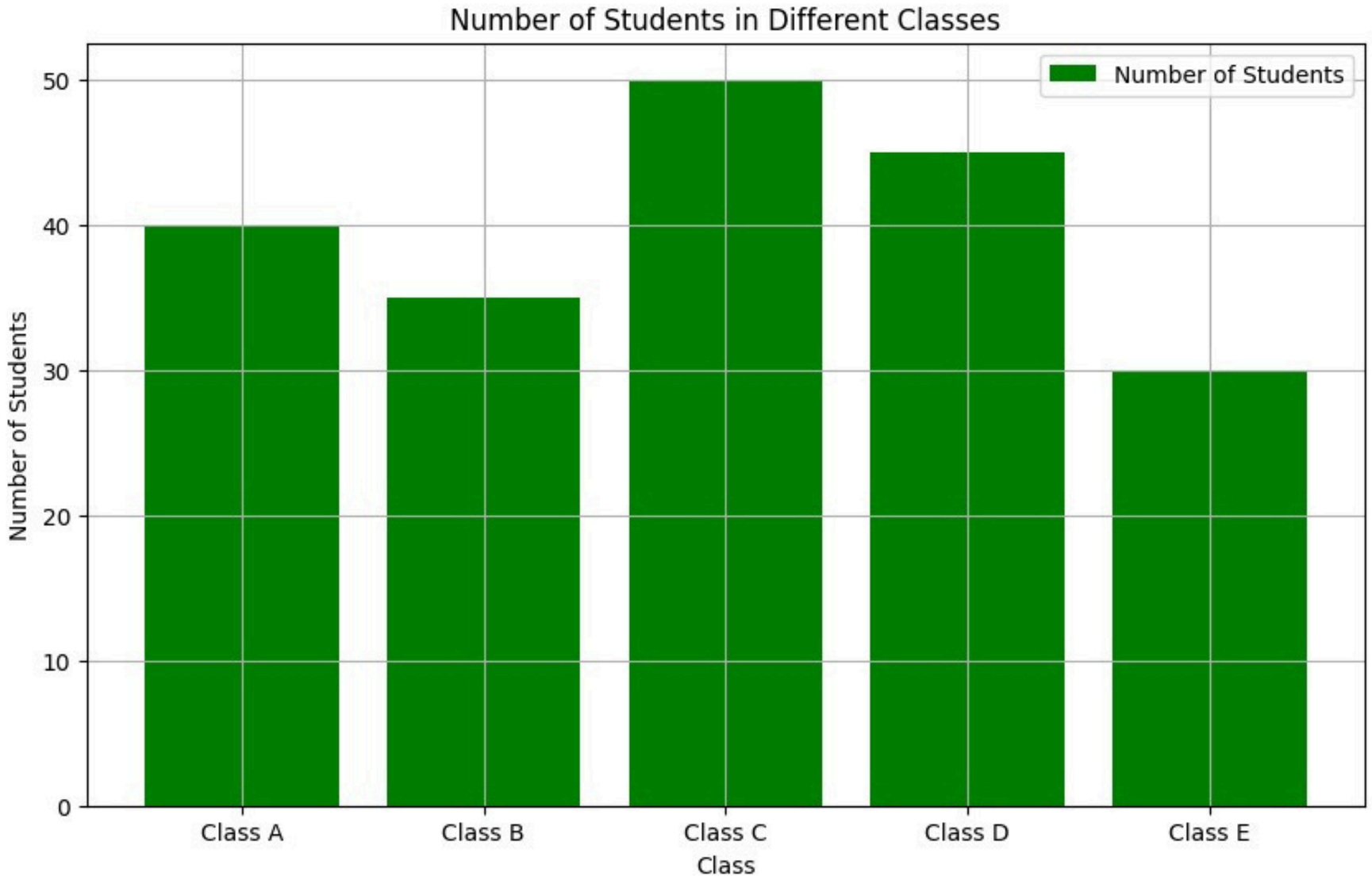
A bar plot or bar chart is a chart that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent.

In [ ]:

```
# Bar Plot
# Parameters: x (x-axis labels), height (height of the bars), label (label for the bars), color (color of the bars)
# Returns: A bar plot

# Example: Number of students in different classes
classes = ['Class A', 'Class B', 'Class C', 'Class D', 'Class E']
students = [40, 35, 50, 45, 30]

plt.figure(figsize=(10, 6))
plt.bar(classes, students, label='Number of Students', color='green')
plt.xlabel('Class')
plt.ylabel('Number of Students')
plt.title('Number of Students in Different Classes')
plt.legend()
plt.grid(True)
plt.show()
```



Histogram

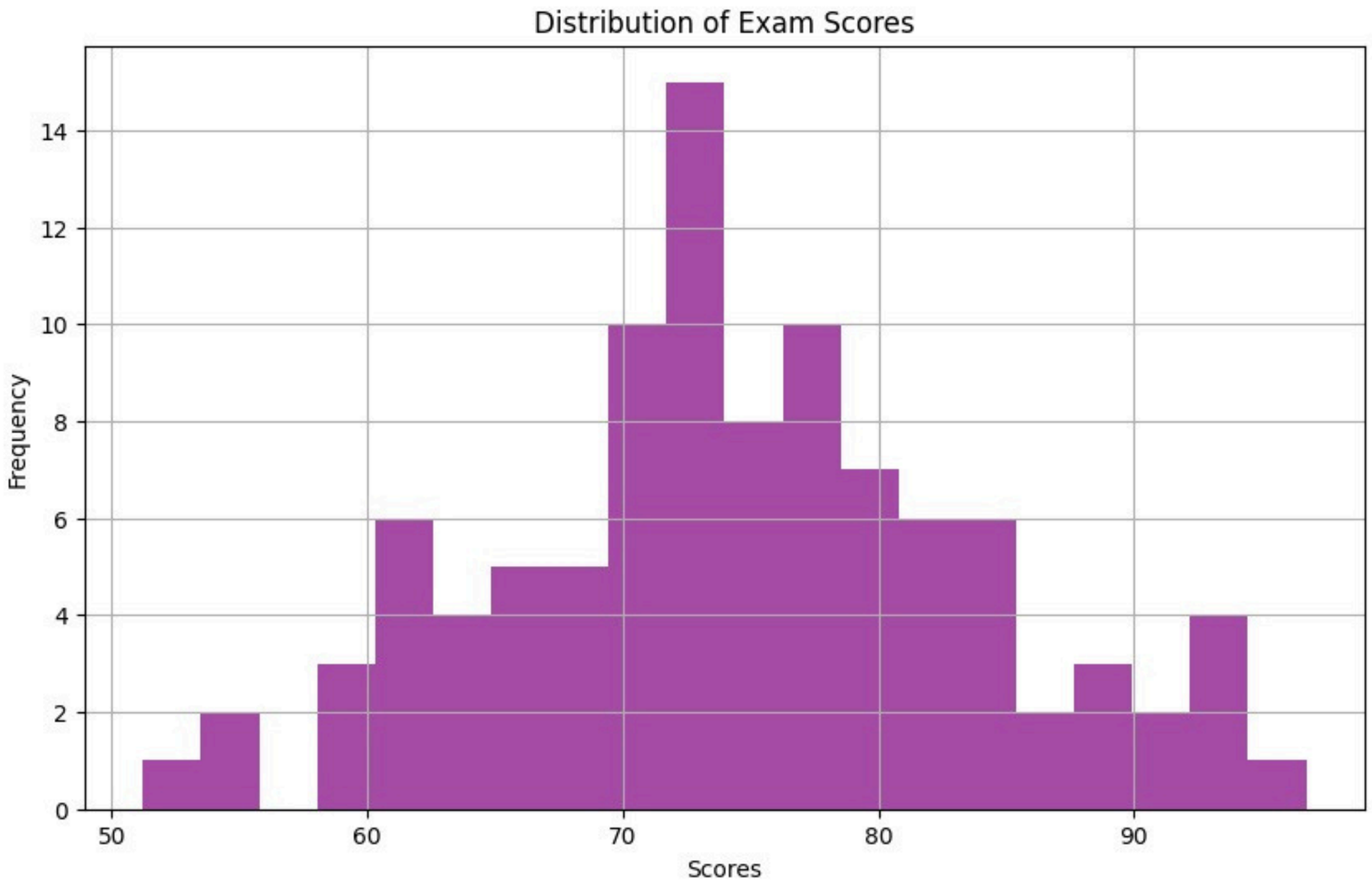
A histogram is an accurate representation of the distribution of numerical data. It is an estimate of the probability distribution of a continuous variable.

In [ ]:
```
# Histogram
# Parameters: x (data values), bins (number of bins), color (color of the bars), alpha (transparency level)
# Returns: A histogram

# Example: Distribution of exam scores
scores = np.random.normal(75, 10, 100)

plt.figure(figsize=(10, 6))
```

```
plt.hist(scores, bins=20, color='purple', alpha=0.7)
plt.xlabel('Scores')
plt.ylabel('Frequency')
plt.title('Distribution of Exam Scores')
plt.grid(True)
plt.show()
```



Advanced Plotting

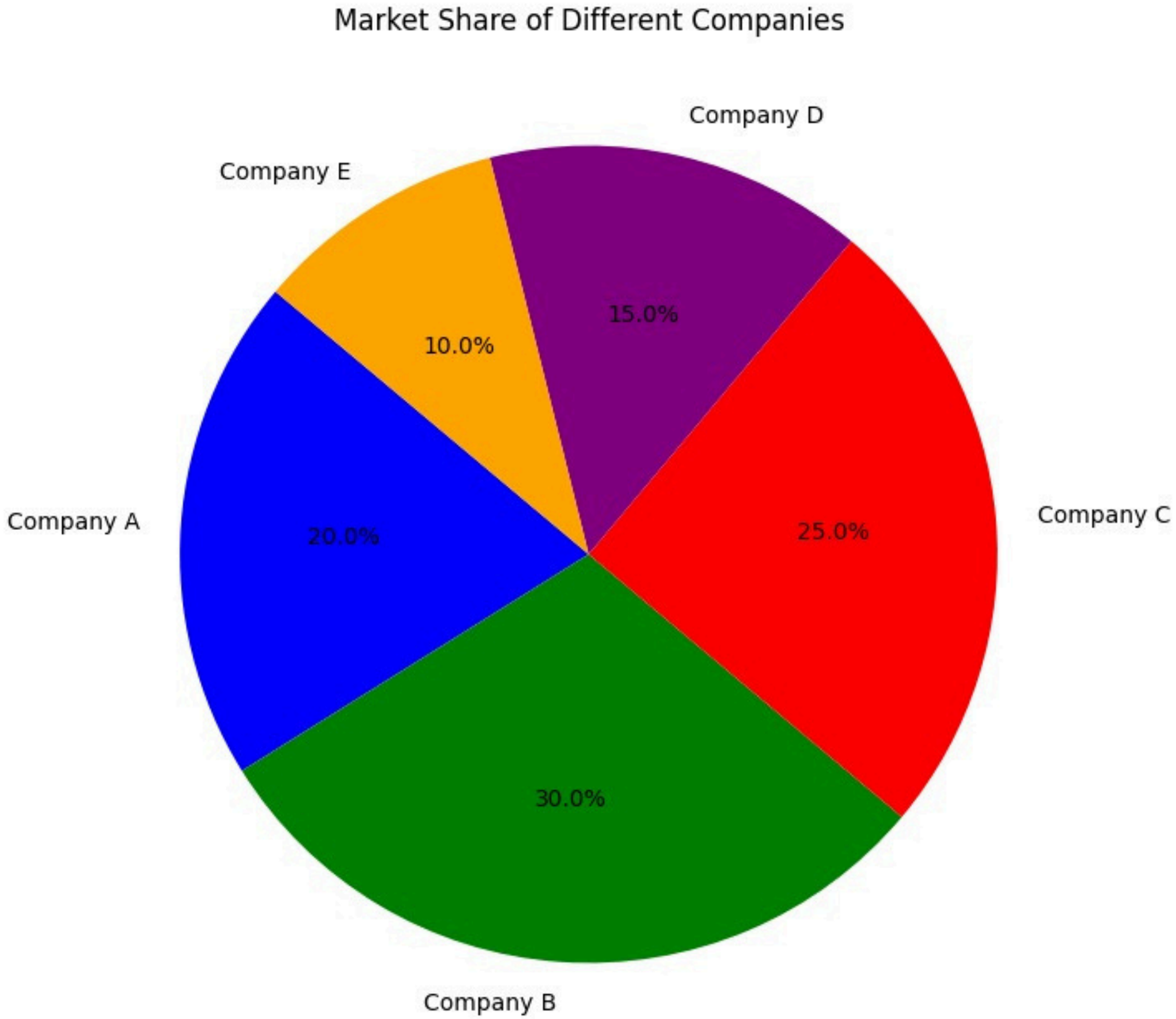
Pie Chart

A pie chart is a circular statistical graphic, which is divided into slices to illustrate numerical proportions.

In [ ]:
```
# Pie Chart # Parameters: x (sizes of the slices), labels (labels of the slices), colors (colors of the slices), autopct (string or function to label the wedg # Returns: A pie chart


# Example: Market share of different companies
sizes = [20, 30, 25, 15, 10]
labels = ['Company A', 'Company B', 'Company C', 'Company D', 'Company E']
colors = ['blue', 'green', 'red', 'purple', 'orange']

plt.figure(figsize=(8, 8))
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=140)
plt.title('Market Share of Different Companies')
plt.show()
```



Box Plot

A box plot (or box-and-whisker plot) shows the distribution of data based on a five-number summary: minimum, first quartile, median, third quartile, and maximum.

In [ ]:
```
# Box Plot # Parameters: x (data values), labels (labels for each dataset), notch (whether to use notched boxes),
patch_artist (for filled boxes style), box
```

```
# Returns: A box plot

# Example: Distribution of salaries in different departments
data = [np.random.normal(50000, 10000, 100) for _ in range(5)]

plt.figure(figsize=(10, 6))
plt.boxplot(data, labels=['Dept A', 'Dept B', 'Dept C', 'Dept D', 'Dept E'], notch=True,  patch_artist=True, boxprops={'facecolor': 'lightgreen'}
plt.xlabel('Department')
plt.ylabel('Salary')
plt.title('Distribution of Salaries in Different Departments')
plt.grid(True)
plt.show()
```



Heatmap

A heatmap is a graphical representation of data where individual values are represented as colors.

In [ ]:
```
# Heatmap
# Parameters: data (2D dataset), cmap (colormap), annot (whether to annotate cells)
# Returns: A heatmap

# Example: Heatmap of student scores across subjects
data = np.random.randint(50, 100, size=(10, 5))
subjects = ['Math', 'Science', 'English', 'History', 'Geography']
students = ['Student ' + str(i) for i in range(1, 11)]
df = pd.DataFrame(data, columns=subjects, index=students)

plt.figure(figsize=(12, 8))
sns.heatmap(df, annot=True, cmap='viridis')
plt.title('Heatmap of Student Scores Across Subjects')
plt.show()
```



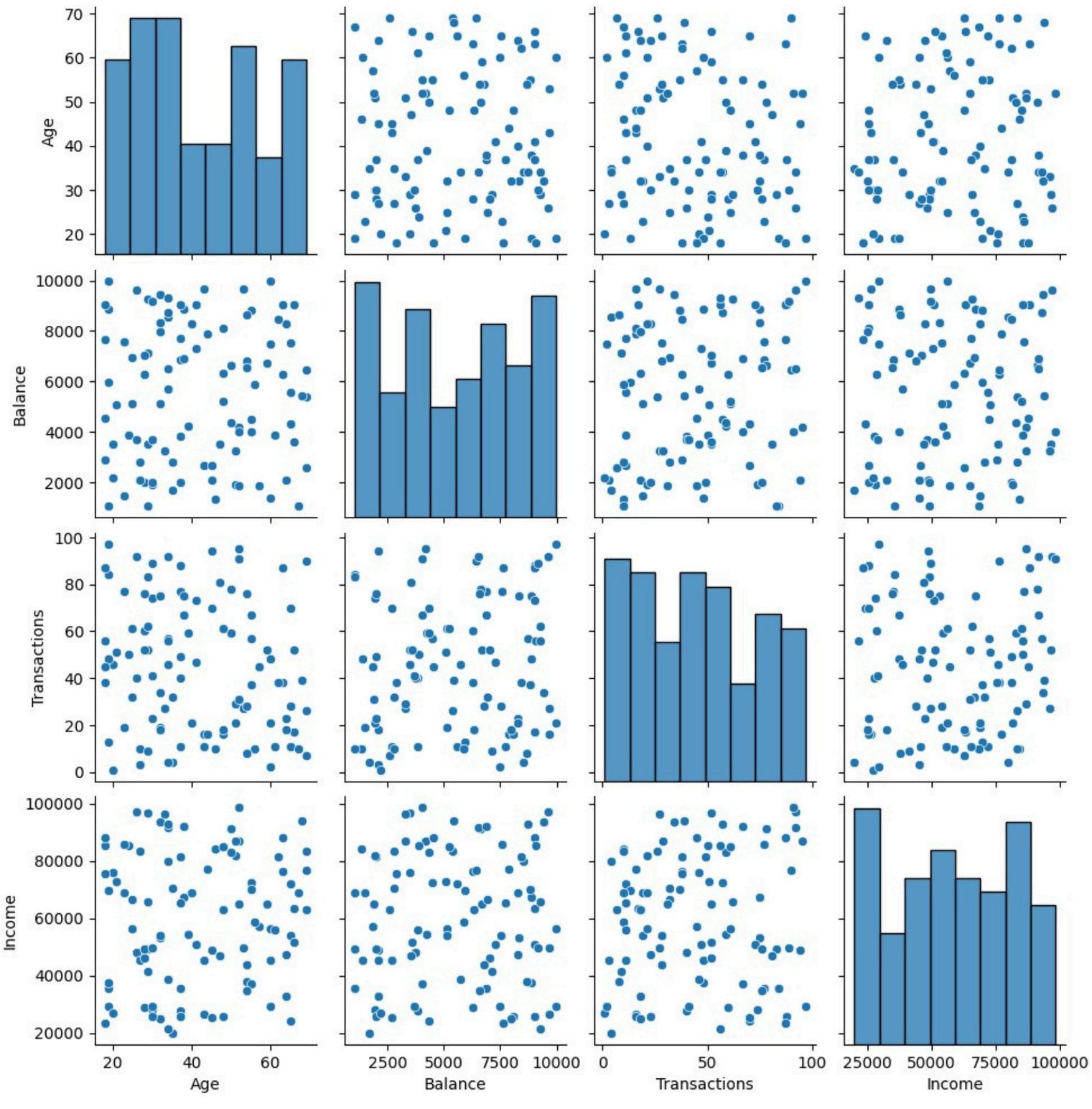
Pairwise Charts

In [ ]:
```
# Pairwise Charts Example # Parameters: data
(DataFrame with numerical columns)
```

```
# Returns: Pairwise plots for all numerical columns in the DataFrame

# Example: Pairwise charts for banking data
bank_data = pd.DataFrame({
    'Age':     np.random.randint(18,     70,     100),
    'Balance': np.random.randint(1000,  10000,  100),
    'Transactions': np.random.randint(1,  100,  100),
    'Income': np.random.randint(20000, 100000, 100)
})

sns.pairplot(bank_data)
plt.show()
```

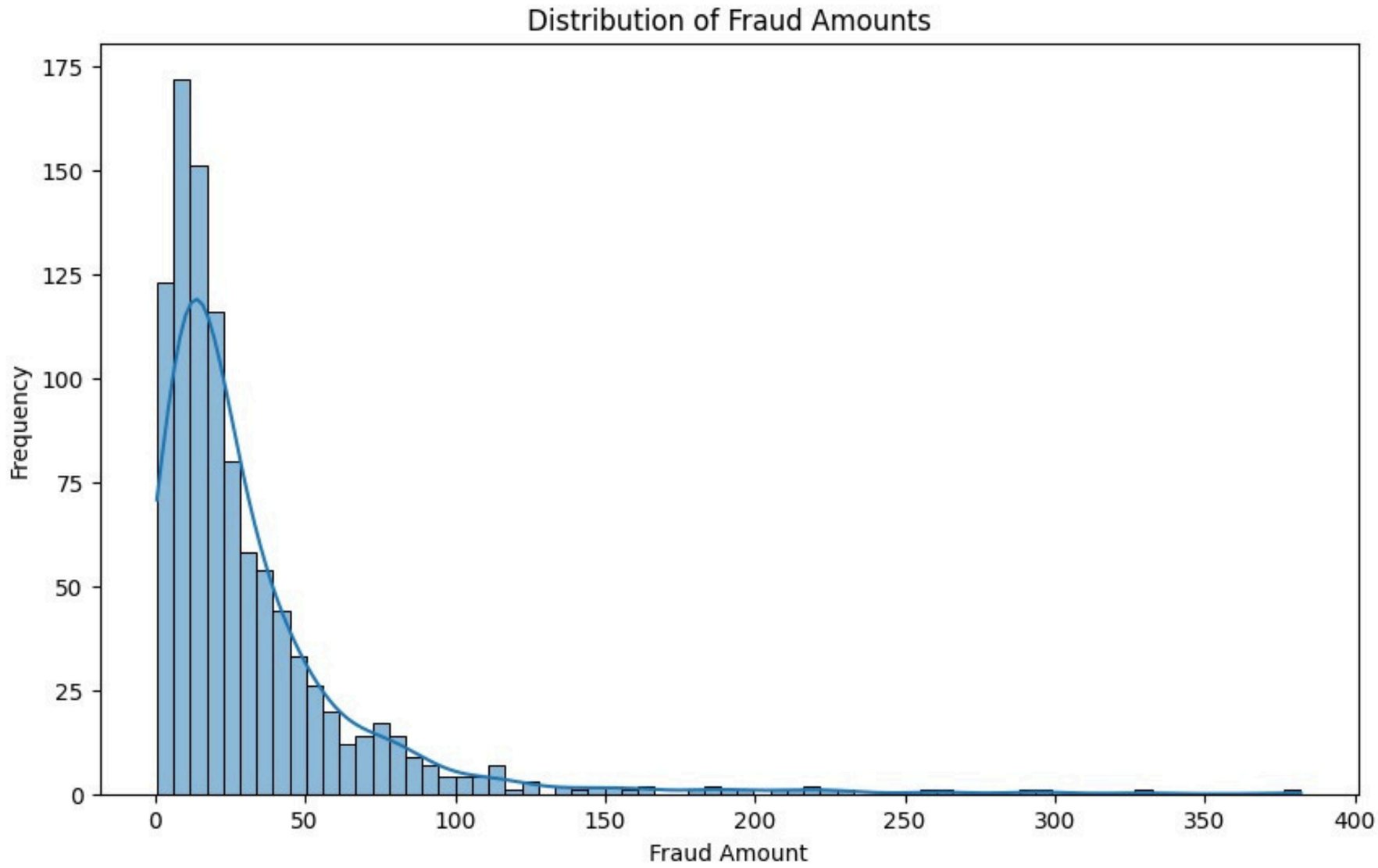

Age
Balance
Transactions
Income
20
30
40
50
60
70
2000
4000
6000
8000
10000
0
20
40
60
80
100
20000
40000
60000
80000
100000
20
40
60
2500
5000
7500
10000
0
50
100
25000
50000
75000
100000

Statistical Distributions

In [ ]:
```
# Statistical Distribution Example
# Parameters: data (array of data values)
# Returns: Histogram and KDE plot


# Example: Distribution of fraud amounts
fraud_amounts = np.random.lognormal(mean=3, sigma=1, size=1000)

plt.figure(figsize=(10, 6))
sns.histplot(fraud_amounts, kde=True)
plt.xlabel('Fraud Amount')
plt.ylabel('Frequency')
plt.title('Distribution of Fraud Amounts')
plt.show()
```

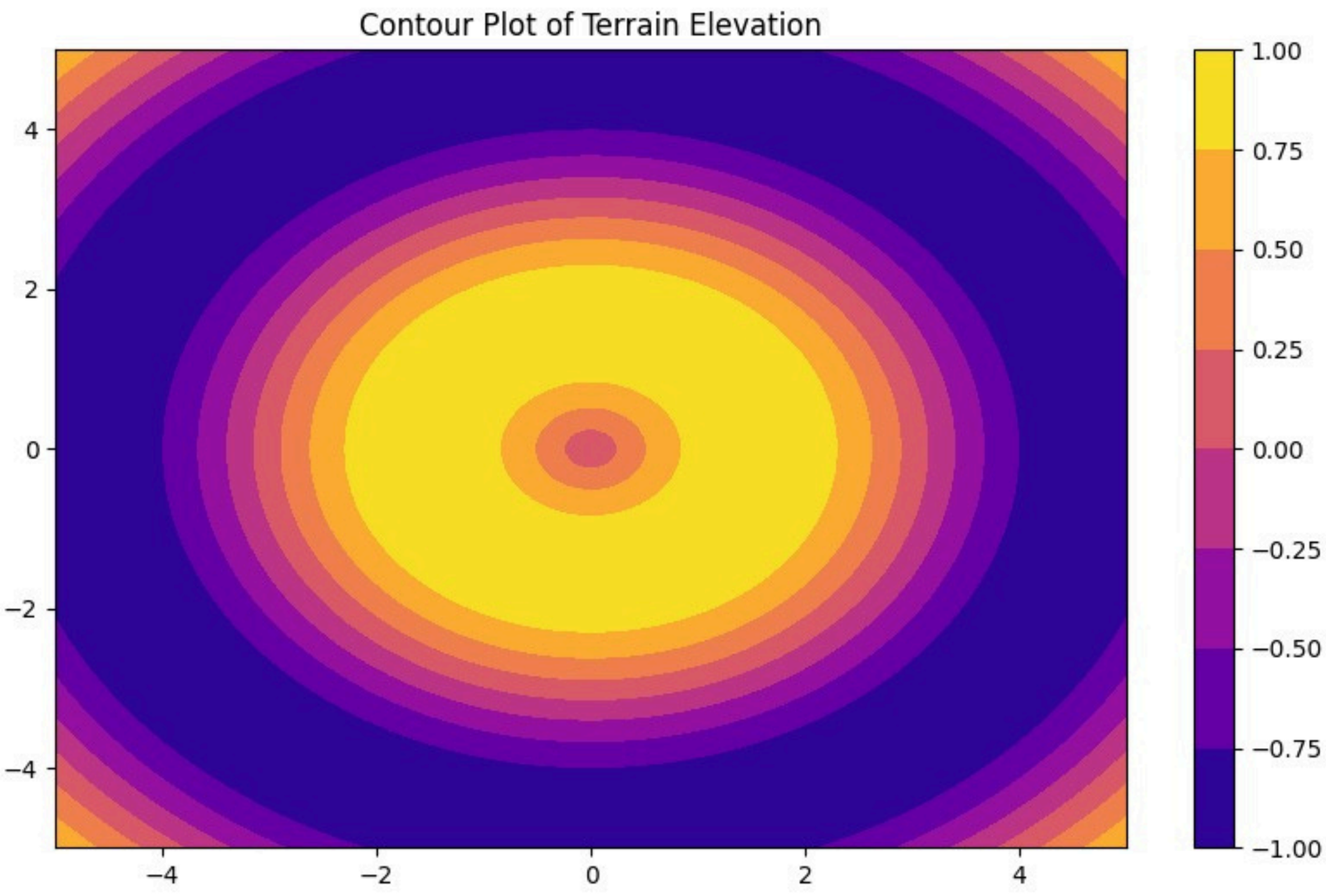


Gridded Charts

In [ ]:
```
# Gridded Charts Example
# Parameters: X, Y, Z (data values)
# Returns: Contour plot of gridded data


# Example: Contour plot of terrain elevation
x = np.linspace(-5, 5, 100) y =
np.linspace(-5, 5, 100) X, Y =
np.meshgrid(x, y) Z = np.sin(np.sqrt(X**2 +
Y**2))
```

```
plt.figure(figsize=(10, 6))
plt.contourf(X, Y, Z, cmap='plasma')
plt.colorbar()
plt.title('Contour Plot of Terrain Elevation')
plt.show()
```



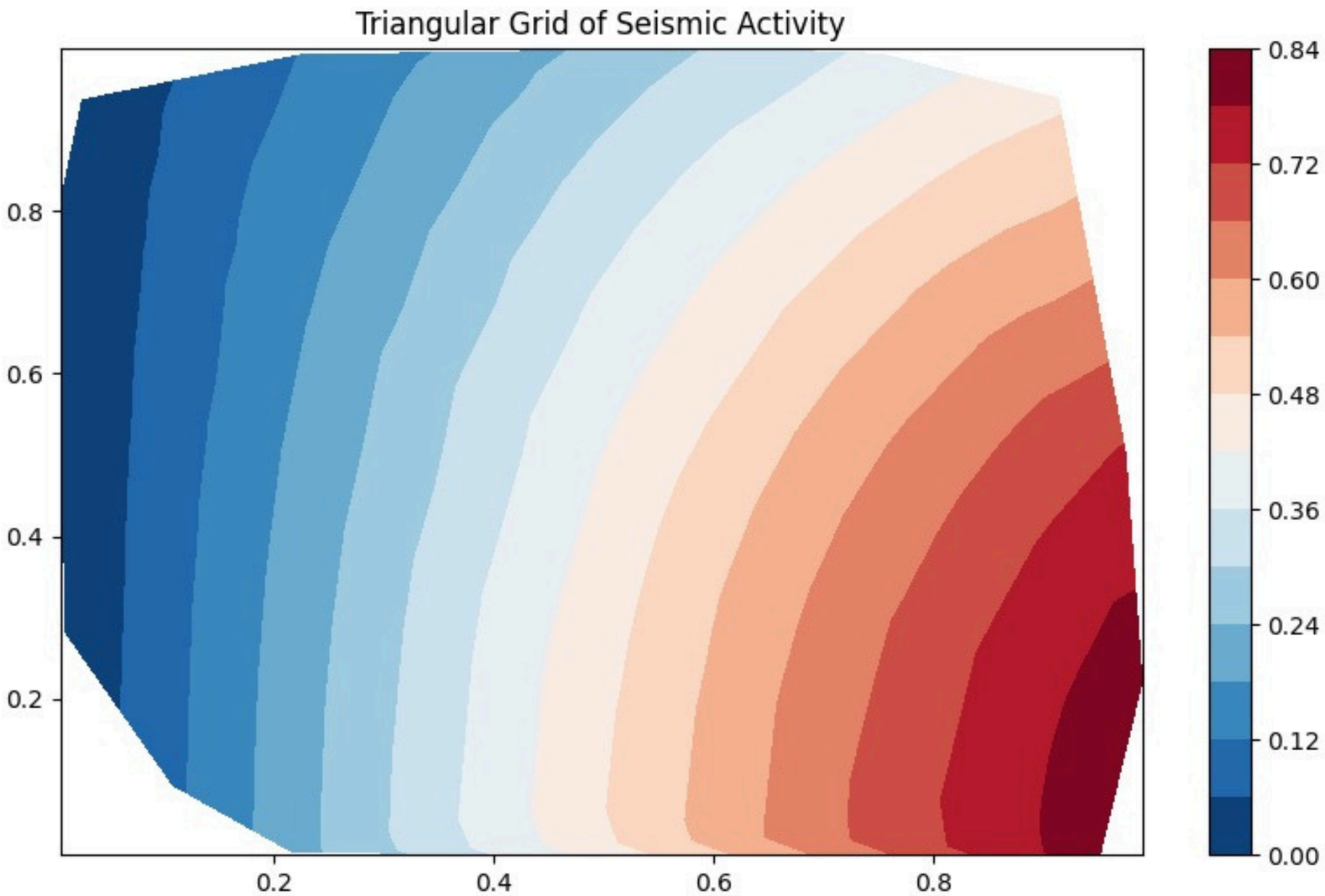
Irregularly Gridded Charts

In [ ]:
```
# Irregularly Gridded Charts Example
# Parameters: x, y, z (data values)
# Returns: Triangular grid of irregularly spaced data

# Example: Triangular grid of seismic activity
```

```
x = np.random.rand(100) y
= np.random.rand(100) z =
np.sin(x) * np.cos(y)

plt.figure(figsize=(10, 6))
plt.tricontourf(x, y, z, levels=14, cmap='RdBu_r')
plt.colorbar()
plt.title('Triangular Grid of Seismic Activity')
plt.show()
```



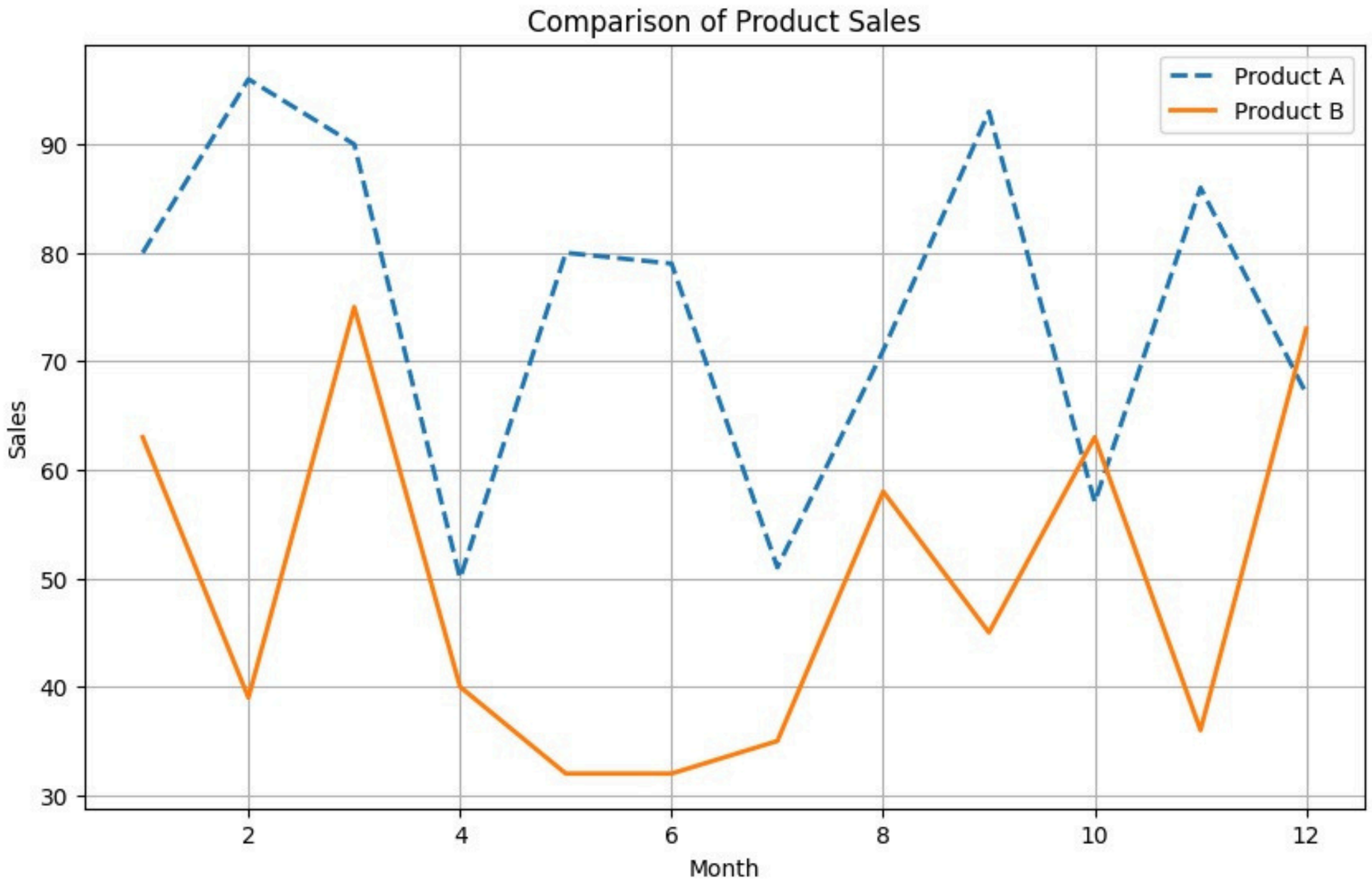
Styling and Legends

In [ ]:

```
# Styling Example
# Parameters: x, y (data values), linestyle (style of the line), linewidth (width of the line)
# Returns: A styled plot

# Example: Comparison of product sales
months = np.arange(1, 13)
product_a_sales = np.random.randint(50, 100, 12)
product_b_sales = np.random.randint(30, 80, 12)

plt.figure(figsize=(10, 6)) plt.plot(months, product_a_sales, label='Product A',
linestyle='--', linewidth=2) plt.plot(months, product_b_sales, label='Product B',
linestyle='-',      linewidth=2)      plt.xlabel('Month')      plt.ylabel('Sales')
plt.title('Comparison of Product Sales') plt.legend() plt.grid(True) plt.show()
```



Multiple Subplots

In [ ]:
```
# Multiple Subplots Example # Parameters: fig,
axs (figure and axes objects) # Returns: A figure
with multiple subplots

# Example: Subplots of different metrics
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

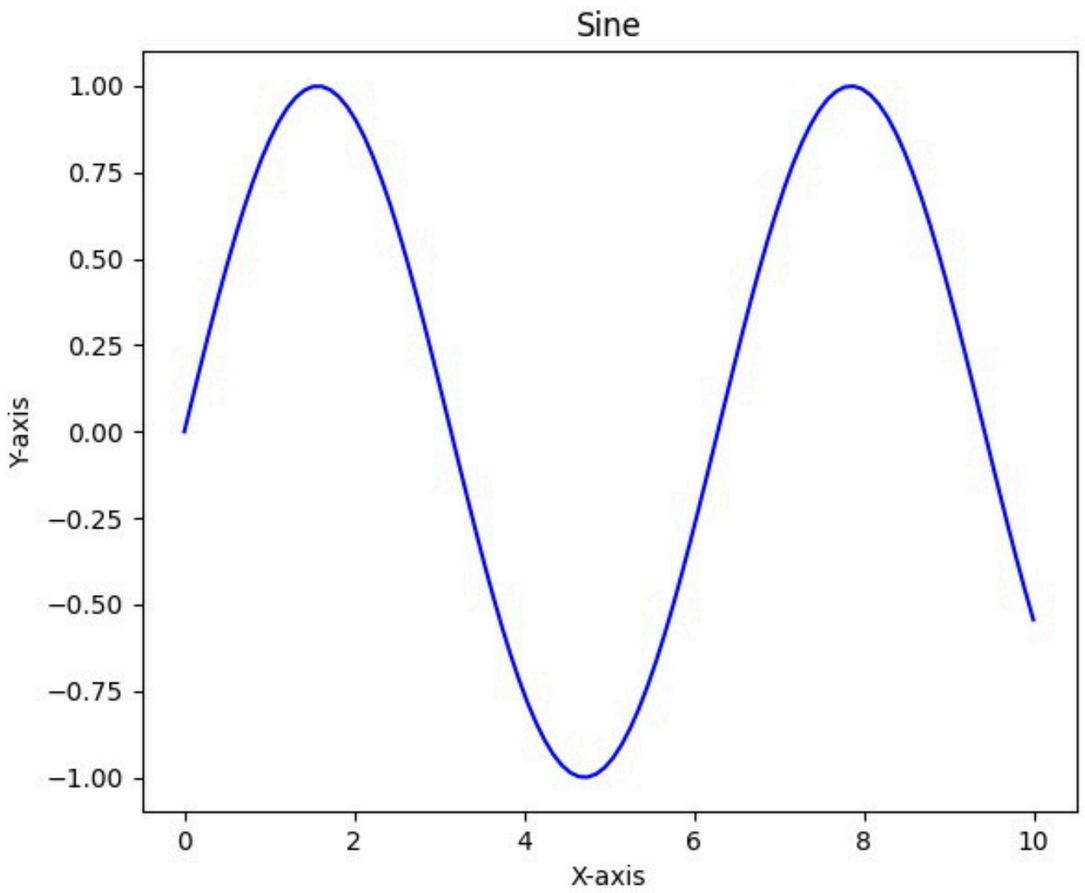
x = np.linspace(0, 10, 100)
y1   =   np.sin(x)   y2   =
np.cos(x)
```

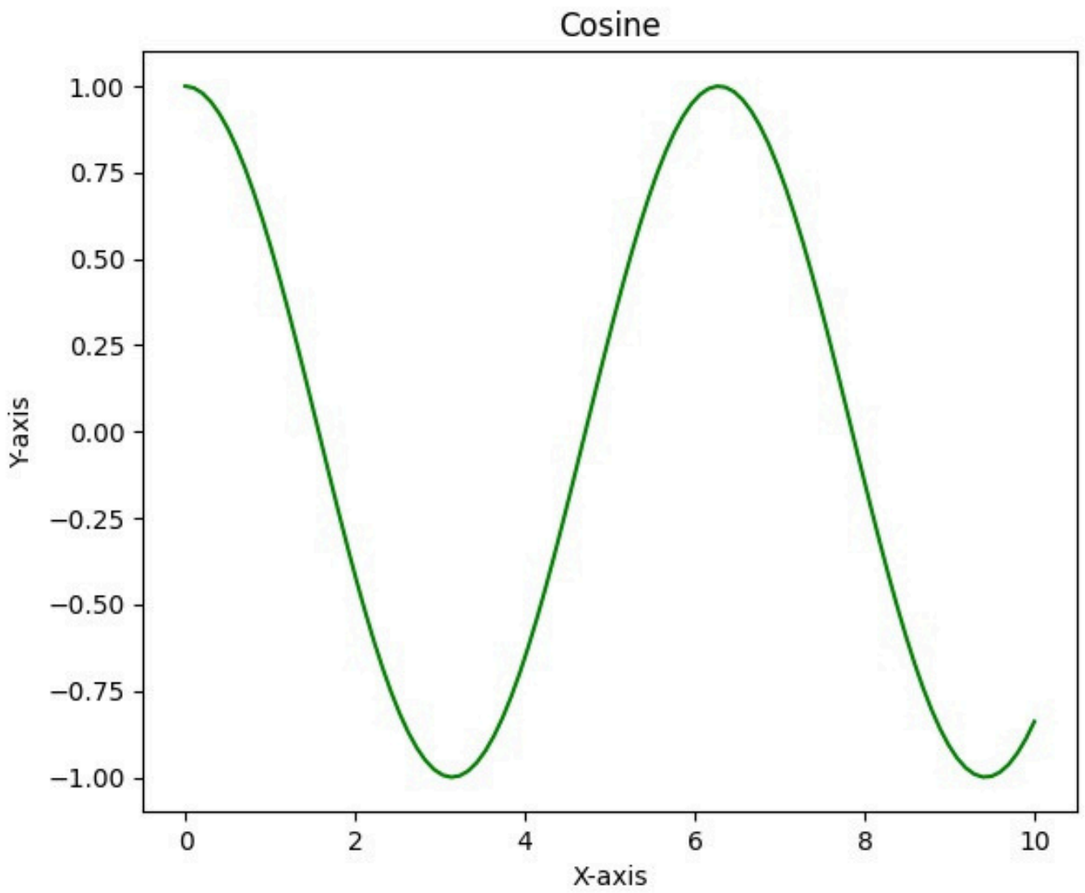
```
y3 = np.tan(x)
y4 = np.exp(x / 10)

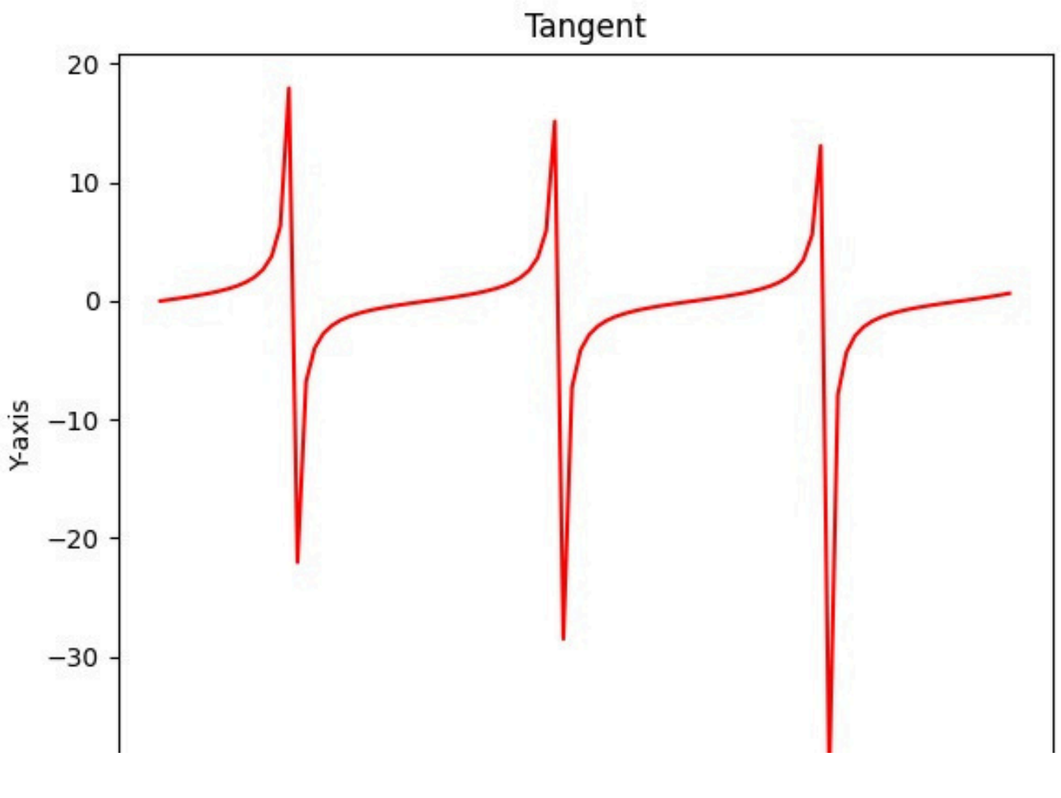
axs[0, 0].plot(x, y1, color='blue')
axs[0, 0].set_title('Sine')
axs[0, 1].plot(x, y2, color='green')
axs[0, 1].set_title('Cosine')
axs[1, 0].plot(x, y3, color='red')
axs[1, 0].set_title('Tangent')
axs[1, 1].plot(x, y4, color='purple')
axs[1, 1].set_title('Exponential')

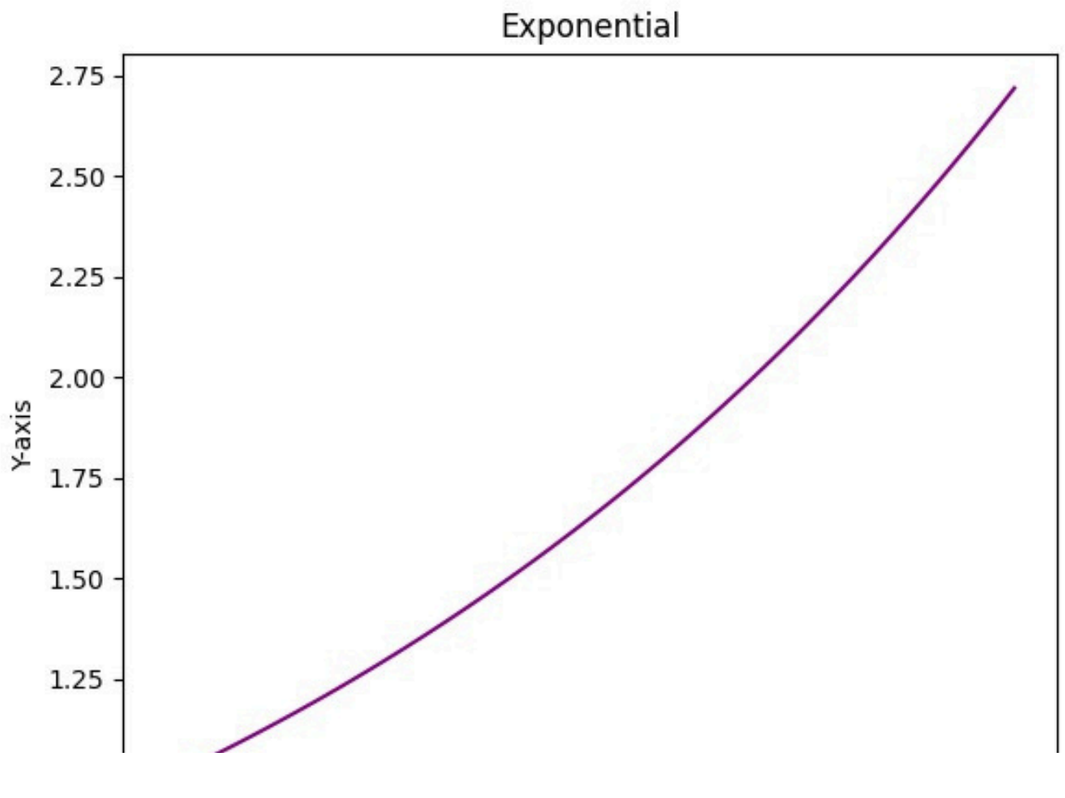
for ax in axs.flat:
    ax.set(xlabel='X-axis', ylabel='Y-axis')

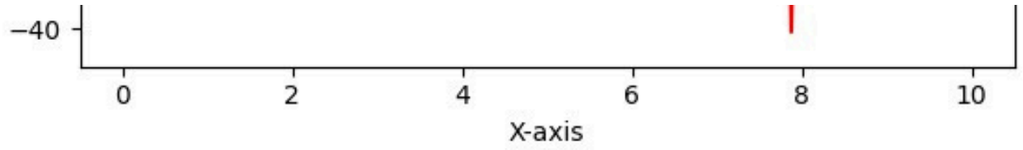
plt.tight_layout()
plt.show()
```

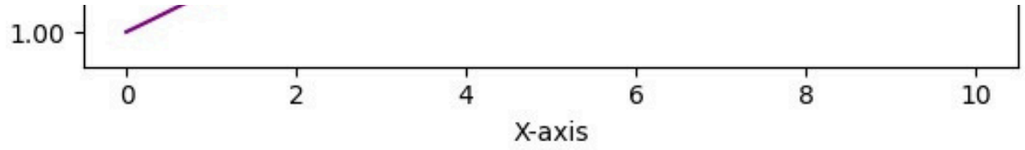

Sine
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Y-axis
0
2
4
6
8
10
X-axis


Cosine
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Y-axis
0
2
4
6
8
10
X-axis


Tangent
20
10
0
−10
−20
−30
Y-axis


Exponential
2.75
2.50
2.25
2.00
1.75
1.50
1.25
Y-axis





Multiple Charts in Grid

In [ ]:
```
# Multiple Charts in Grid Example
# Parameters: fig, ax (figure and axes objects)
# Returns: A figure with multiple charts

# Example: Multiple metrics in a single chart
fig, ax = plt.subplots(figsize=(12, 8))

x = np.linspace(0, 10, 100)
y1   =   np.sin(x)   y2   =
np.cos(x) y3 = y1 + y2 y4 =
y1 - y2

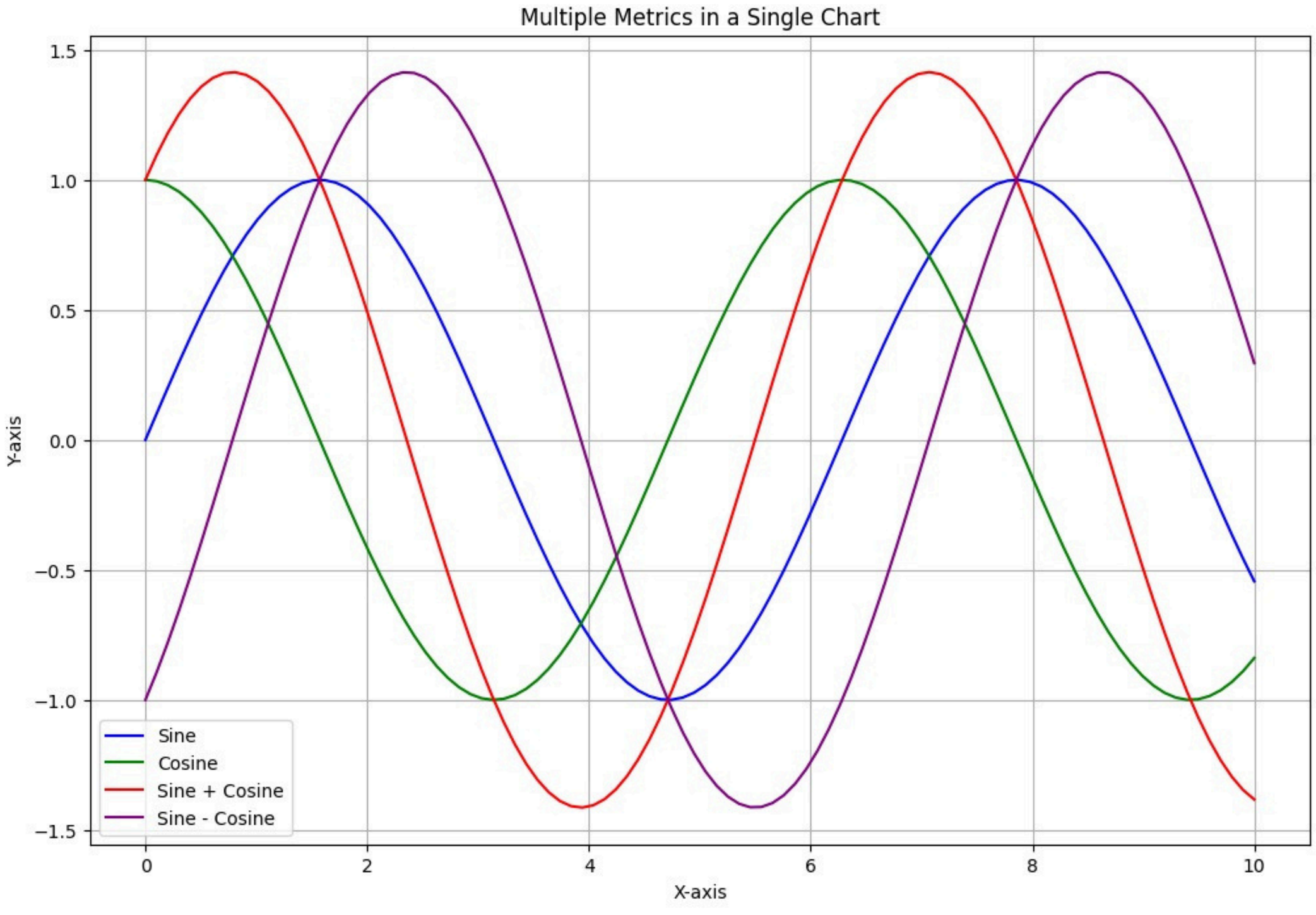


ax.plot(x, y1, label='Sine', color='blue')
ax.plot(x, y2, label='Cosine', color='green')
ax.plot(x, y3, label='Sine + Cosine', color='red')
ax.plot(x, y4, label='Sine - Cosine', color='purple')

ax.set_xlabel('X-axis')     ax.set_ylabel('Y-axis')
ax.set_title('Multiple Metrics in a Single Chart')
ax.legend() ax.grid(True)



plt.show()
```



Animated Charts

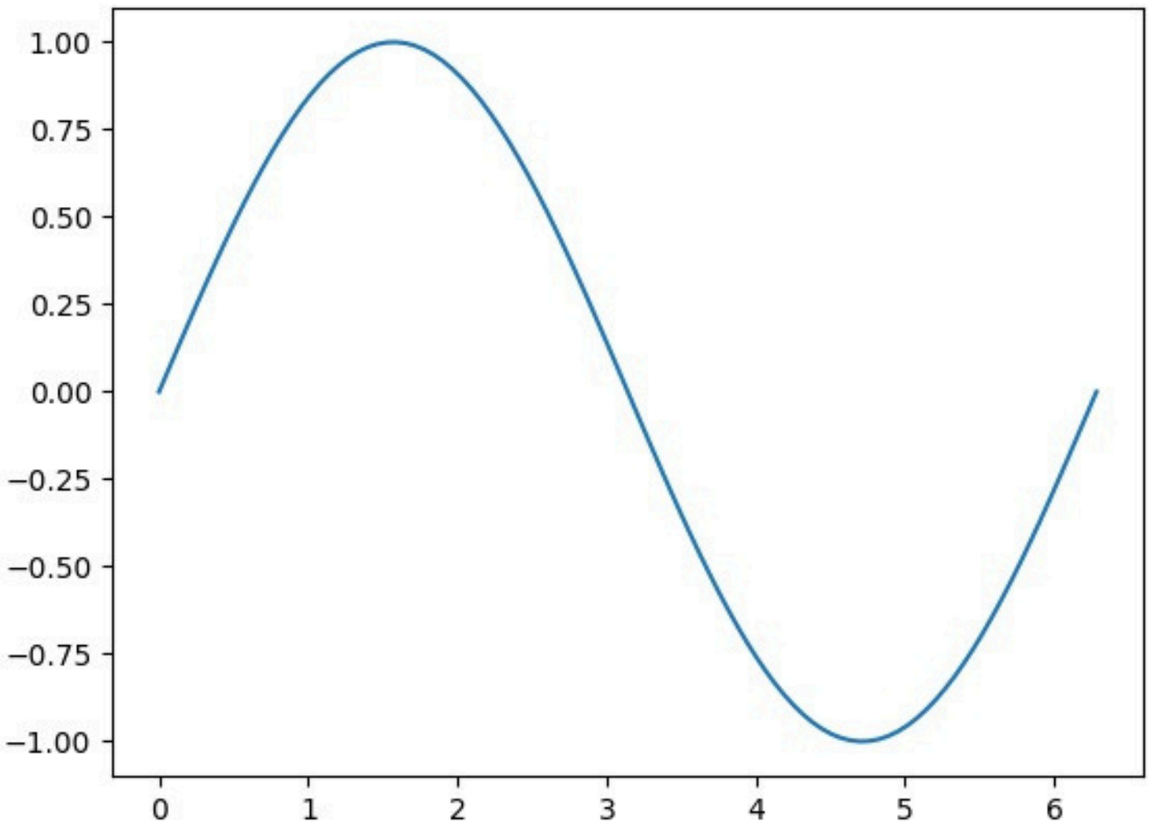
In [ ]:
```python
# Animated Chart Example
# Parameters: i (frame number)
# Returns: An animated sine wave

# Example: Animated sine wave
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()
x = np.linspace(0, 2 * np.pi, 100)
line, = ax.plot(x, np.sin(x))

def update(i):
    line.set_ydata(np.sin(x + i / 10))  # Update the data.
    return line,

ani = FuncAnimation(fig, update, frames=100, blit=True)
plt.show()
```



Follow me on Sumit Khanna for more updates